
天津大学

计算机视觉

实验 2、图像分割实验报告



学 院 智能与计算学部

专 业 计算机科学与技术

学 号 3022244006

姓 名 樊建琦

计算机视觉实验二：图像分割

樊建琦

1. 语义分割算法

1.1. 语义分割网络简述

此次实验的网络结构如 Fig.1所示，下采样部分使用 ResNet 作为 encoder。在上采样阶段，我们采用了 U-Net 的解码器结构，先用转置卷积把通道数降到原先的半数，再连接相对应的 encoder，以获得丰富的上下文信息，同时将通道数翻倍，最后用双层卷积将通道数落回半数，同时先后自乘通道注意力的权重和空间注意力的权重。

backbone 选用 ResNet 有多方面的考量：首先，ResNet 通过残差连接有效解决了深层网络的梯度消失问题，使我们能够利用更深的网络层次提取更丰富的语义信息；其次，ResNet 在 ImageNet 上的预训练权重提供了优秀的迁移学习基础，加速了模型收敛；再其次，ResNet 的多尺度特征提取能力非常适合语义分割任务的需求。

1.2. 网络结构

1.2.1. encoder

ResNet 的 four layers 结构与语义分割问题十分契合，image 在 resize 为 (224,224) 之后送入 net 中，先通过初始化的卷积层将 RGB 的三层特征升维到 64，之后开始进入 four layers 的结构不断地下采样获得特征信息。同时在 net 中选择了空洞卷积来谋求更大的“免费的”感受野。

上采样阶段保留了 unet 的转置卷积以及 skip connection 思想，加入了 CBAM。具体来讲，通道数为 2048 的张量进入上采样阶段后，

1.2.2. decoder 简述

decoder 部分主要思想是转置卷积把通道数先降下来，恢复一倍的图片尺寸，之后考虑上下文信息，将相同通道数的 encoder 层连接上，但是这无法避免的把通道数又升了上去，于是用 DoubleConv 里的卷积层把通道数降下来，这里的 DoubleConv 的设计考虑了几方面需求，首先需要降特征，所以先用一个 (3,3) 的卷积核，零

填充为 1，把特征降到了半数，之后为了凑 (5,5) 卷积的感受野效果，我又添加了一个通道数不变的 (3,3) 卷积核。

1.2.3. DoubleConv

最后在 DoubleConv 里面考虑注意力机制的问题，本质上是门控的问题，一个非常好的软性决策，因为两个注意力矩阵里的值都是 sigmoid 过的，在区间 (0,1) 里的值，不是二值的，所以避免了硬性的选择或者不选择，同时具有自适应的思想，因为这两个权重矩阵是可学习的。

1.2.4. CBAM

下面具体的来看通道注意力和空间注意力机制，考虑一随机张量，其 $size=(1,64,224,224)$ ， $x = x * self.ChannelAttention$ 的过程本质上是一个自乘权重矩阵， $self.ChannelAttention$ 的 size 是 (1,64,1,1)，每个通道得到一个权重的值，这个值可以理解成这个通道有多重要。乘法可以理解成对每个通道的重要性进行加权，也可以理解成“这个通道有多重要”的度量。 $ChannelAttention$ 对每一个通道都展平了，将最有意义的，最有代表性的值保留了下来，并且用这个值代表这个通道，一方面节省计算消耗，另一方面与人类做视觉决策的过程很相似。

空间注意力也是一样的，他本质上把一个位置上的所有通道整合成一个，只关注在 (224,224) 里面的相对位置，不考虑这个处于位置的哪一个层次（哪一个通道），所以考虑 $x = x * self.SpatialAttention(x)$ 的话，size 的变化应该是 (1,64,224,224) 变化到 (1,1,224,224)。

在 senet 的时候，他是选择 average pooling 的，但是我这里就是 max pooling 和 average pooling 都算一遍，之后把这两个值加起来，理论上讲他会变得更理解哪里更重要，这个位置的哪个特征更重要，而且这种决策很软性，通过这种选择性的机制，告诉网络“看哪里”（空间注意力）和“看什么特征”（通道注意力）。

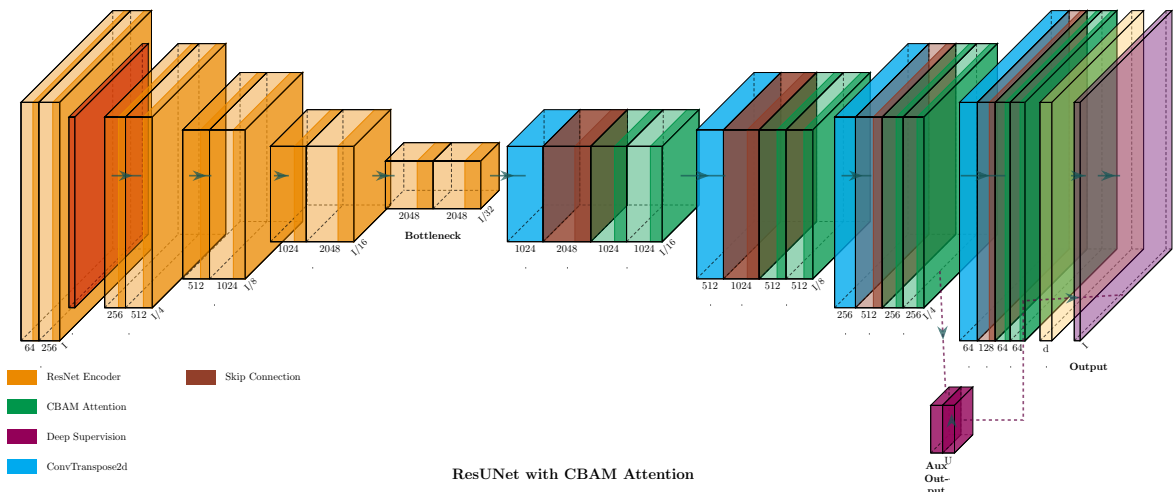


Figure 1. The net work structure.

1.3. 组合损失函数

一开始没有考虑组合损失函数，只是用的交叉损失熵函数 (nn.CrossEntropyLoss(ignore_index=255))，但如 Fig.2所示，光有这个损失函数的话，对不平衡数据集不够敏感。所以后来就换成了 CrossEntropyLoss, DiceLoss, FocalLoss 的组合损失函数，权重设置为 [0.5, 0.3, 0.2]。

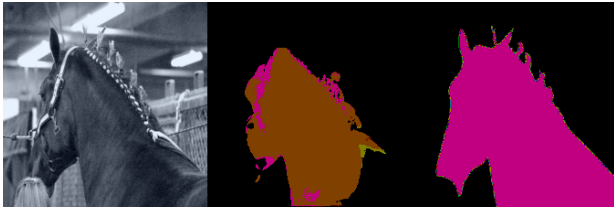


Figure 2. 单独用交叉损失熵的弊端

1.4. 超参数设置情况以及实验设置细节

如 Fig.7所示，后续实验中讨论了初始 learning rate 在 $1e-2$, $1e-3$, $1e-4$, $1e-5$ 的情况下，metrics 的结果，其中 encoder 选用的 ResNet50, optimizer 选用的 SGD(optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_decay=1e-4)), scheduler 选用的余弦退火 (optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30, eta_min=1e-5))，损失函数用的组合损失函数 (class CombinedLoss)。

如 Fig.??所示，后续实验中讨论了 ResNet50 和 ResNet152 相比，基本收敛的情况下，额外的深层架构能否带来更高的 mIoU 收益，其中参数设置如下：learning rate 在 $1e-3$ 的情况下，metrics 的结果，其中 encoder 选用的 ResNet50, optimizer 选用的 SGD(optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_decay=1e-4)), scheduler 选用的余弦退火 (optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=30, eta_min=1e-5))，损失函数用的组合损失函数 (class CombinedLoss)。

2. 结果分析

2.1. 定性描述可视化结果

我们先定性分析三个不同场景下的语义分割结果，我会在附录中添加 16 个可视化结果。

如 Fig.3所示，第一个是小孩坐在沙发上，因为小孩和沙发接触的非常紧密，所以在边界处很难分清像素的分类、归属。其次，手和沙发的把手有很多穿插的细节，很难做到把连续纠缠的两个类别区分开，但是这里的结果定性看上去除了毛边没有恢复细节，其他的都很好。



Figure 3. 小孩坐在沙发上的可视化结果定性分析

如 Fig.4所示，第二个是狗和小孩挤在车的后座上，首先小孩被包裹在襁褓里，很难分清清楚小孩子的类别，其次，狗和小孩贴

的很密集，很难将二者区分开，但是这里的结果定性看上去除了襁褓的左连接没有恢复细节，其他的都很好。



Figure 4. 狗和小孩挤在车的后座上的可视化结果定性分析

如 Fig.5所示，第三个是单独的类别，摩托有很多的毛边，很难分清周围每一个像素的类别，但是这里的结果定性看上去除了车尾的细毛边没有恢复细节，其他的都很好。



Figure 5. 带有毛边的摩托的可视化结果定性分析

2.2. 定量描述可视化结果



Figure 6. 上述可视化结果的定量指标

如 Fig.6所示，这次的测试比较轻量级，mIoU 只到了，本次实验最高时用 ResNet152，在 epoch=170 的时候达到了最高 mIOU=59.31%。其训练的 loss 如 Fig.8所示。

2.3. 不同超参设置下对实验结果的影响情况

第一个实验，探究初始 learning rate 在 $1e-2$, $1e-3$, $1e-4$, $1e-5$ 的情况下，metrics 的结果，其结果如 Fig.7所示 可以很清楚的看到初始学习率对实验结果的影响是巨大的， $1e-4$ 的时候甚至刚在 20 个 epoch 的时候略有起效就过拟合了，一部分原因和选取的优化器有关，SGD 主要的迭代思想如公式所示，这里迭代的 w 其实是一个很复杂，很多元的 metrics， a 是学习率乘以梯度， g 其实是带有噪音 η 的，以 w 为自变量的函数。

$$w_{k+1} = w_k * a_k \cdot g(w_k, \eta) \quad (1)$$

第二个实验，探究了网络的深度对 mIOU 的增效是否明显，第一个网络是 ResNet50, 第二个网络是 ResNet152, 通过 Fig.8的 train loss 可以看到在经历了 40, 60, 140, 三个重要拐点之后，loss 开始趋于平缓，并且保持这种特性到 epoch=300，相邻 loss 的差小于给定值，所以我们认为收敛了。可以看到在 epoch=170 的时候达到了最大的 mIoU=59.31%。但在性能和训练时间的 tradeoff

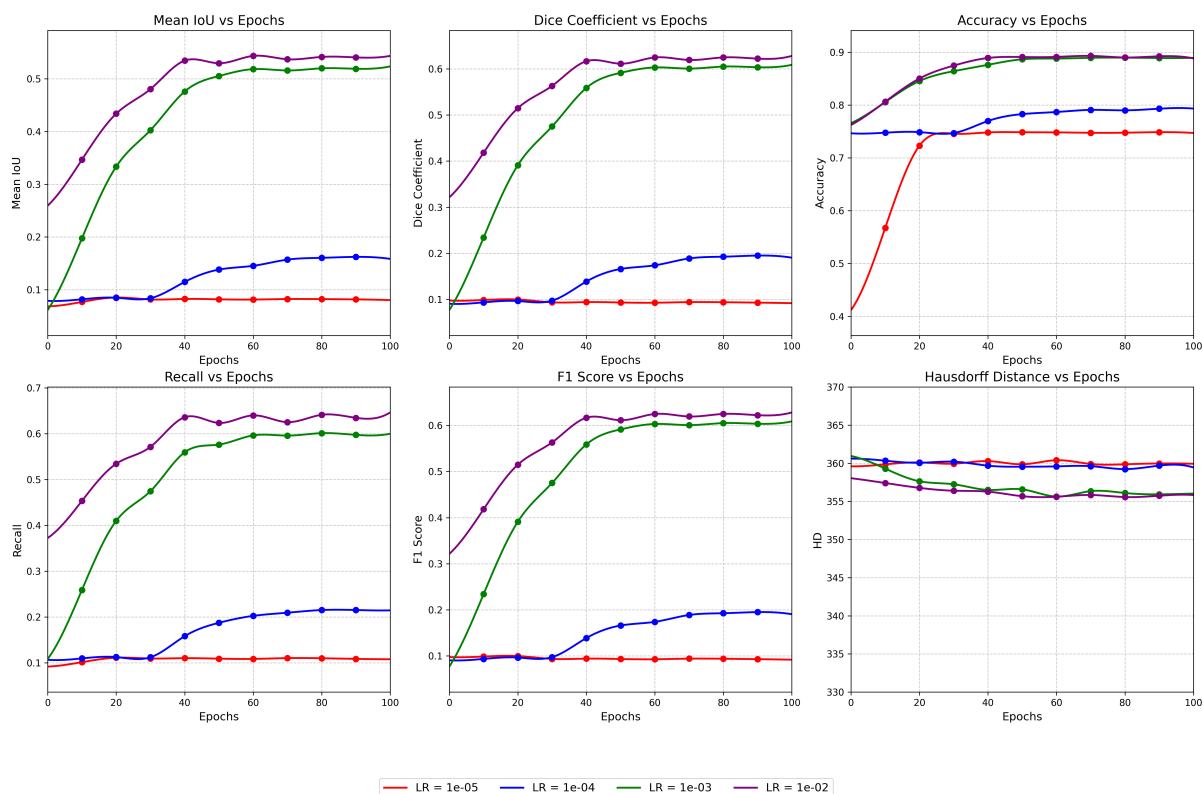


Figure 7. 探究不同初始学习率对 mIoU, Dice, Accuracy, Recall, F1, HD

上, 可见 152 层的 resnet 并没有占据优势, 一方面是训练时间翻倍的增长, 另一方面是 mIoU 只是略微的增长, 相比 50 层的 ResNet 而言, 仅仅增长了 5 个点以内。

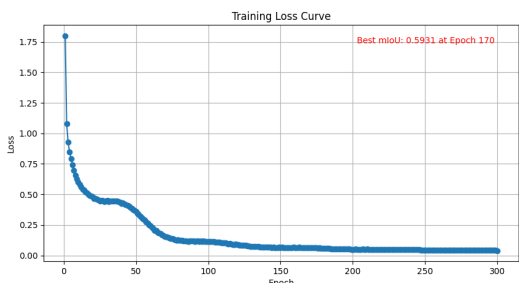


Figure 8. encoder=ResNet152, epoch=300

我们现在看 50 层的 ResNet 的效果, 如 Fig.9所示的 train loss 结果, 可以看到拐点出现的 epoch 基本上是一致的, 而且基本都是在 epoch=140 的时候开始达到峰值, 之后开始收敛。

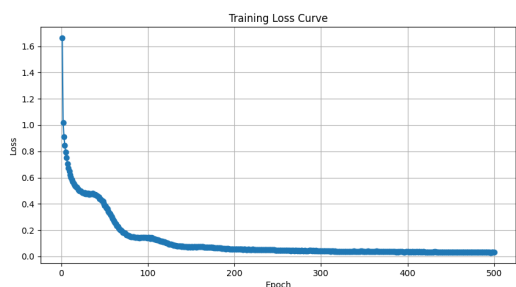


Figure 9. encoder=ResNet50, epoch=500

2.4. 调参规律

首先, 学习率调度器和优化器都按照 1.4 设置, 初始学习率最好是 $1e-2$, encoder 用 ResNet, 层数越高效果肯定越好, 但是时间代价有些不值当, 另外损失函数必须用 1.3 所示的组合损失函数。epoch 基本上在 200 以内一定到达峰值。随着初始 lr 的降低, mIoU 会走下坡路, encoder 是 vgg 也可以, 但是由于缺少了 residual connection, 所以性能略差。batch size 在 8 和 16 都还可以。

3. 总结

这次实验相当复杂, 从一开始 dataloader 构建之间的 split 开始就有难度了, 后面助教老师给了 split 之后, dataloader 那里面的 resize 的插值算法也有说法, 否则 gt 会传进去 float 当作 label。网络结构基本上就是拿老师在答疑课上讲的 unet, 之后把 backbone 换成 resnet, 因为 vgg 太轻量级了, 我试了很多次, 也加了 CBAM, 但是感觉 mIoU 还是上不到 50%, 所以就换成 resnet 了, resnet50 其实训练起来很快。

加上 ResNet 之后又在 DoubleConv 里面加了 CBAM, 这里面用的还是双层注意力, 就是把 avg pooling 和 max pooling 都算出来, 之后加在一起, 以前没用过这种办法, 但是这次试了发现确实有效, 这个想法一开始是出于看助教老师给的 DoubleConv, 他那里有一个先把通道数降到想要的数量了, 之后又来了一个 (outchannel, outchannel) 的卷积, 后来我发现这么做也挺有效果的, 相当于拿了免费的感觉野, 之后我就想到把同样的东西垒到一起会不会有奇效呢? 但是后来查文献的时候发现这个方法已经有人发过文章证明了。其实已经用了很多很多次 senet 这种 squeeze, excitation 的思想了, 但是每一次用还是能有不一样的体会。包括

ResNet 也是。

画网络结构是一个很复杂的事情，一开始确实打算用 draw.io 或者 visio 画了得了，但是后来感觉 PlotNeuralNet 要好看一点，就 clone 了他们的项目，看了他们 U-Net 是怎么画的，之后在 U-Net 的基础上画了我的 Net，但是我一开始打算让这个网络结构也呈现 U-Net 论文里面那个能拐弯的样子的，我后来发现是可行的，虽然网上没有这种样子的成品，但是只要把 east 改成 southeast，他就可以在下采样的时候往右下走，改走线就可以了。但是我到现在还有一个问题，不知道是不是 PlotNeuralNet 的 bug，如果只画一个 rightbox，也必须得是 width=a，之后 xlabel="c","”，这么写，不能把 xlabel 这里面用一个花括号写，我把我的 PlotNeuralNet 源码附在智慧树的 handin 里面了。

感谢王正扬助教老师，一方面为我们提供了十分受用的答疑，另一方面提供了 split 和 visualize 的源码，这部分虽然不在网络的范围内，但是非常的繁琐。希望这门课越来越好！